

Design Document

Project Description

Spot is a full-stack workout accountability app, built for the CS5610 Web Development class at Northeastern University. Most people don't quit the gym because they lack a plan; they quit because nobody notices when they skip. Spot is built around accountability between friends: it turns skipping a workout into something that has a visible, shared cost, so quitting quietly stops being an option.

There are three main areas:

1. **Log Workout & Session History:** Where users record and review their training. Users log a session with a date, one or more exercises (name, sets, reps, weight), and optional notes. The app automatically flags a new personal record (PR) whenever a logged lift beats the user's previous best for that exercise. The history page lets users browse past sessions, filter them by exercise and date range, edit or delete records, and see their PR-highlighted lifts at a glance.
2. **Pacts (Dashboard):** Where users form ongoing accountability agreements with a partner. A pact is a shared weekly target; for example, "we each train three times a week." Each week the app counts both partners' logged sessions against the target; the week only clears, and the shared streak advances, if both partners hit it. If one person falls short, the streak resets for both. Users can create a pact with a partner, view the pact dashboard with the current streak and each partner's progress, edit the target, or dissolve the pact.
3. **Challenges:** Where users post and take on one-off dares. A challenge is a public, time-boxed dare like "100 push-ups a day for 7 days." Another user accepts it and logs daily proof entries during the window; the challenge resolves as completed or failed (and fails immediately on a missed day). Challenges are grouped into Open, Accepted, and Done so users can see what's available to take on and what they're currently committed to.

The app is built with React (Hooks, client-side rendering) on the frontend, with Node.js and Express on the backend and MongoDB (native driver) for storage. It uses four MongoDB collections; Users, Sessions, Challenges, and Pacts; each supporting full CRUD. The collections are linked: sessions and challenges store the `_id` of the user they belong to, and pacts store the `_ids` of the two partners, so the accountability logic always points back to real users and their real training data. Authentication is handled with Passport so every session, pact, and challenge belongs to a real account.

The goal of this project was to build something where data in one place has real consequences elsewhere, where logging (or not logging) a workout directly decides whether a shared streak lives or dies, while demonstrating a full-stack web application with a real database, a REST API, and a client-side-rendered React frontend.

User Personas/Stories

Persona 1: The Fading New Year's Lifter

Name: Danny Osei, 27 **Role:** Office worker who restarts the gym every year **Location:** Columbus, OH

Background: Danny starts strong every January and quits by March, every year. It's never the workout plan that fails him ; it's that no one notices or cares when he skips, so skipping has no cost. He wants a way to make quitting quietly impossible: a pact with his roommate where a missed week visibly breaks a shared streak for both of them, so his slacking actually lets someone else down.

Scenario: Danny searches for his roommate by username and starts a pact with a target of three workouts a week. Over the week he logs his sessions, and checks the pact dashboard to see both his and his roommate's progress. When he sees his partner is behind, he texts him ; because if either of them misses, the streak they've been building resets for both.

User Stories:

1. As Danny, I want to register and log in securely, so my workout history and pacts are private to my account.
2. As Danny, I want to search for another user by username, so I can find my partner and start a pact.
3. As Danny, I want to create a pact with a partner and a weekly session target, so we're both committed to a shared goal.
4. As Danny, I want to view my pact dashboard with our shared streak and both partners' progress this week, so I can see if either of us is falling behind.
5. As Danny, I want to edit a pact's weekly target or dissolve a pact, so our commitment can change as our schedules change.

Persona 2: The Competitive Gym Friend

Name: Alisha Rahman, 24 **Role:** Grad student who turns everything into a game **Location:** Boston, MA

Background: Alisha loves competition and thrives on friendly stakes. A plain workout log bores her; what motivates her is a dare with a clear win-or-lose outcome and friends watching. She

wants to throw down challenges, take on the ones her friends post, and have a definitive completed-or-failed result at the end of each one.

Scenario: Alisha posts a challenge; "100 push-ups a day for 7 days"; for her friends to see. A friend accepts one of her older challenges, and Alisha accepts theirs in return. Each day of the window she logs a proof entry; the day she misses one, the challenge flips to failure, and she knows she has to start a new one to redeem herself.

User Stories:

1. As Alisha, I want to create a challenge with a description and a time window, so my friends can accept it.
2. As Alisha, I want to accept an open challenge and log proof entries during its window, so the challenge ends as completed or failed.
3. As Alisha, I want to see challenges grouped by status (open, accepted, done), so I can tell what's available to take on and what I'm currently committed to.

Persona 3: The PR Chaser

Name: Rohan Mehta, 28 **Role:** Serious lifter focused on progressive overload **Location:** Jersey City, NJ

Background: Rohan trains seriously and cares about measurable progress. He logs every session in detail and wants to know the moment he sets a new personal record, instead of scrolling back through months of entries to compare weights by hand. He wants a complete, accurate training history and automatic recognition when he hits a new best.

Scenario: After a heavy session, Rohan logs each exercise with its sets, reps, and weight. The app immediately flags his bench press as a new PR since it beat his previous best. Later he filters his history to just squats over the last two months to review his progression, and fixes a session he'd entered with the wrong weight so his record stays accurate.

User Stories:

1. As Rohan, I want to log a workout session with the date, exercises, sets, reps, weight, and notes, so I have a complete training history.
2. As Rohan, I want the app to flag a new personal record when I log a heavier lift, so my progress is visible automatically.
3. As Rohan, I want to view my session history filtered by date range and exercise, so I can review my recent training.
4. As Rohan, I want to edit or delete a session I logged incorrectly, so my history stays accurate.

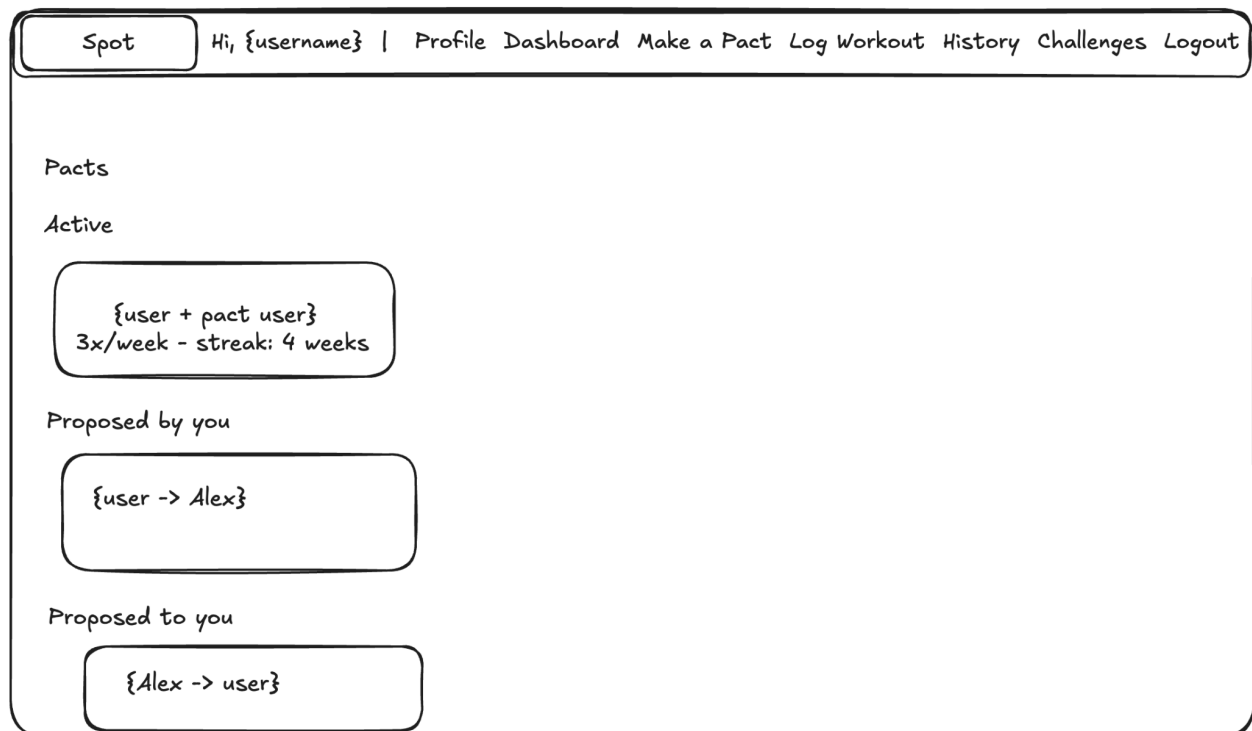
Division of Work

Nipun ; Sessions, PRs & Challenges (Full Stack): Log Workout page and Session History page (browse, filter by exercise and date range, edit/delete), automatic PR detection, and the Challenges page (create, browse, accept, log proof, and the open to accepted to completed/failed lifecycle with fail-fast). All Express CRUD routes for the Sessions and Challenges collections, the weekly pact-clearing logic that counts each partner's sessions against the pact target and updates the shared streak, and the seed data for sessions and challenges.

Khush ; Users, Auth & Pacts (Full Stack): Passport authentication (register, login, logout, session management) and route protection, user profiles and partner search, and the Pacts dashboard (create a pact with a partner and target, view the dashboard, edit, dissolve). All Express CRUD routes for the Users and Pacts collections, and the seed data for user and pact records.

Design Mockups

Home / Dashboard page: a top nav bar (Spot brand on the left; a greeting and links to Dashboard, Find a Partner, Profile, Log Workout, History, Challenges, and Logout on the right). The dashboard shows the user's pacts as a list of cards, each displaying the partner, the weekly target, and the current shared streak, with an empty state prompting the user to find a partner if they have no pacts yet.



Profile Page: the nav bar, then a "Profile" heading followed by the user's login identity (their @username, shown but never editable) and a "Member since" date. Below that is an editable profile form — Display name, Email, Bio, and Favorite gym, each labeled input pre-filled with the current value, with a "Save changes" button. Next is a "My challenges" section showing the challenges this user has accepted, split into Accepted (still in progress) and Done (completed or failed) lists. At the bottom, a "Delete account" button prompts the user to confirm, then deletes the account and returns them to the login page.

Spot

Hi, {username} | Profile Dashboard Make a Pact Log Workout History Challenges Logout

Profile

Display name

Email

Bio

Favorite gym

SAVE CHANGES

My Challenges

Accepted

Done

Log Workout page: the nav bar, then a single form. A date picker at the top, followed by an "Exercises" section with a row of inputs (exercise name, sets, reps, weight) and an "Add exercise" button that appends more rows, then a notes field and a "Log workout" button. After a successful log, a result panel below the form lists the logged exercises and highlights any that

were new personal records with a "New PR" badge.

Spot Hi, {username} | Profile Dashboard Make a Pact Log Workout History Challenges Logout

Log a Workout

Date

Exercises

Exercise Sets Reps Weight

add exercise

Notes

optional

Session History page: the nav bar, then a row of filter controls (exercise name search, from-date, to-date) above a list of session cards. Each card shows the session date, its exercises (sets by reps at weight, with PRs highlighted), and notes, plus Edit and Delete controls. Editing a

card reveals an inline notes field with Save and Cancel.

Spot

Hi, {username} | Profile Dashboard Make a Pact Log Workout History Challenges Logout

Session History

Exercise

from:

to:

2026-07-04

Bench Press: 3x5 @ 195lbs

Squat: 4x6 @ 225lbs

edit

delete

2026-07-03

Bench Press: 3x5 @ 175lbs

Squat: 4x6 @ 215lbs

edit

delete

Challenges page: the nav bar, then a "Post a challenge" form (description, start date, end date) at the top. Below it, three sections ; Open, Accepted, and Done ; each a scrollable list of challenge cards. A card shows the challenge description, its date window, and a status badge. Open cards have an "Accept" button; accepted cards show a proof-logging form (date, a "completed" checkbox, notes) and the running list of proof entries; done cards show their final

completed/failed status.

Spot

Hi, {username} | Profile Dashboard Make a Pact Log Workout History Challenges Logout

Challenges

Post a challenge

insert challenge

start

end

Open Challenges

{scrollable of open challenges to public}

Accepted Challenges

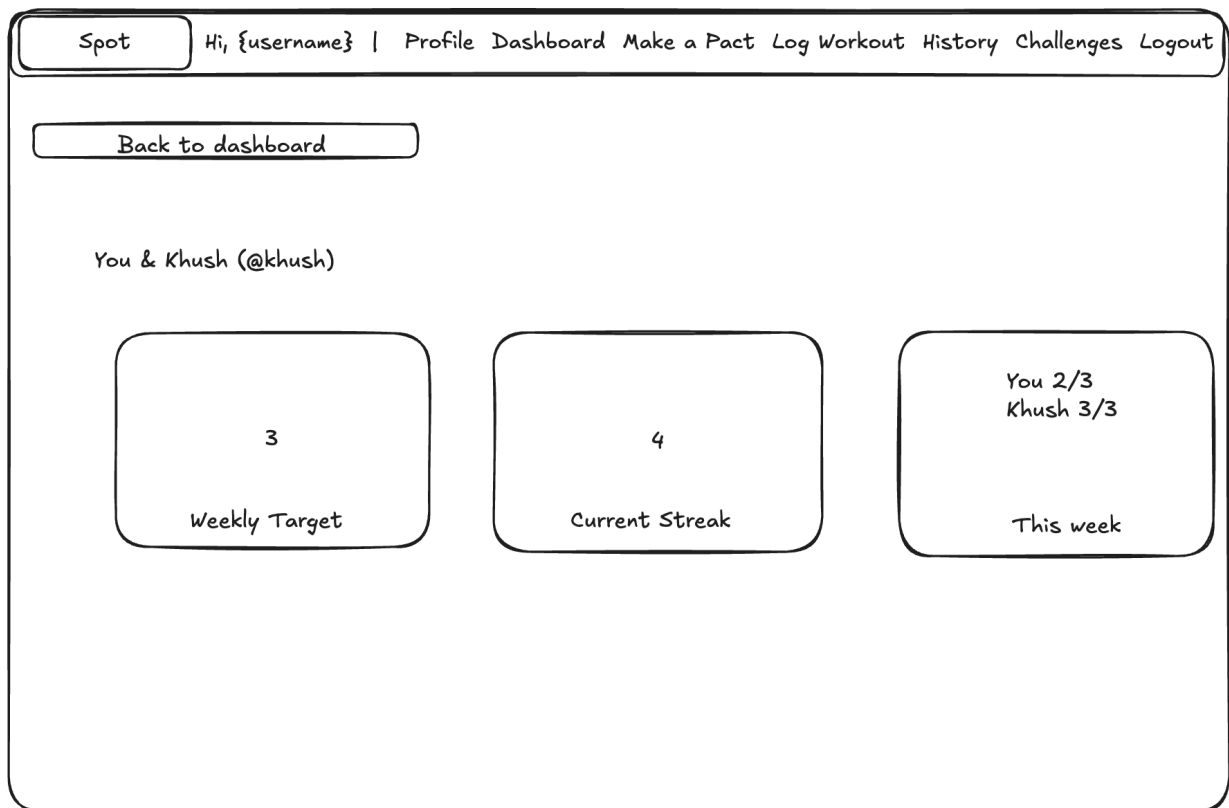
{scrollable of accepted challenges}

Done

{scrollable of finished challenges}

Pact Detail page: the nav bar, then a "← Back to dashboard" link, and a single pact card. The card header shows both partners' names ("You & [partner] (@username)") on the left with a status pill on the right (pending / active). Below the header sit up to three stat tiles side by side: Weekly target (the anchor tile — a plain number when the pact is active, or an editable dropdown with a Save button when the pact is still pending and you were the proposer), Current streak (weeks the pact has cleared, shown only once the pact is active), and This week (each partner's session count against the target this week, labeling your own count as "You," shown only once active). A pact you proposed that hasn't been accepted yet shows a "Waiting to be accepted" note

instead of the streak and this-week tiles. A quiet footer lists both partners' names and emails.



Technology Stack

- **Frontend:** React with Hooks (client-side rendering) + React Router, built with Vite
- **Backend:** Node.js + Express
- **Database:** MongoDB (Native Node.js Driver)
- **Authentication:** Passport (local strategy, session-based) with bcrypt
- **Data Requests:** Fetch API